C Programming Exercises

Activity Sheet 4 — Arrays

Name: PATHIPATI RUTHWIK Roll No: CH.SC.U4CSE25230 Date:13/07/2026

Instructions: Read each program carefully, study the line-by-line explanation, trace through the code by hand, and write the expected output. Then check your answer with the Expected Output provided.

Exercise 1: Weekly Temperature Analyzer

Problem:

Write a C program that reads 7 daily temperatures (in Celsius) into an array. The program should calculate and print the average temperature, and then count how many days had a temperature above the average. Display the average with 2 decimal places and the count of above-average days.

Code:

```
#include <stdio.h>

int main() {
    float temps[7];
    float sum = 0.0, average;
    int i, aboveCount = 0;

    printf("Enter 7 daily temperatures:\n");
    for (i = 0; i < 7; i++) {
        scanf("%f", &temps[i]);
        sum += temps[i];
    }

    average = sum / 7;

    for (i = 0; i < 7; i++) {
        if (temps[i] > average) {
            aboveCount++;
        }
    }

    printf("Average temperature: %.2f\n", average);
    printf("Days above average: %d\n", aboveCount);

    return 0;
}
```

Line-by-Line Explanation:

1. #include <stdio.h> includes the standard input/output library for printf() and scanf() functions.
2. int main() is the program entry point where execution begins.
3. float temps[7]; declares a floating-point array named "temps" with 7 elements. This array will store one temperature value for each day of the week. Array indices range from 0 to 6.

4. float sum = 0.0, average; declares a float variable "sum" initialized to 0.0 (used as an accumulator) and another float variable "average" (uninitialized) to store the computed average later.
5. int i, aboveCount = 0; declares an integer "i" for loop counting and "aboveCount" initialized to 0, which will count how many days exceed the average temperature.
6. printf("Enter 7 daily temperatures:\n"); prompts the user to enter temperature values for all 7 days. The \n moves to the next line after the prompt.
7. for (i = 0; i < 7; i++) { starts a for loop that iterates 7 times (i = 0, 1, 2, ..., 6). This loop reads all temperature values into the array and accumulates their sum simultaneously.
8. scanf("%f", &temps[i]); reads a floating-point number from the keyboard and stores it at position i in the temps array. The & operator provides the memory address of temps[i] so scanf can write to it directly.
9. sum += temps[i]; adds the current temperature value (temps[i]) to the running sum. This is shorthand for sum = sum + temps[i]. After the loop, sum holds the total of all 7 temperatures.
10. } closes the first for loop body. At this point, all 7 temperatures are stored in the array and their total is in sum.
11. average = sum / 7; divides the total sum by 7 (the number of days) to compute the average temperature. Since both sum and 7 are involved in float division, the result retains decimal precision.
12. for (i = 0; i < 7; i++) { starts a second for loop that iterates through the array again. This time, the purpose is to compare each temperature against the computed average.
13. if (temps[i] > average) checks whether the temperature at index i is strictly greater than the calculated average. This conditional determines if the current day was above average.
14. aboveCount++; increments the counter by 1 when the current day's temperature exceeds the average. This accumulates the total number of above-average days.
15. } closes the inner if block.
16. } closes the second for loop. After this loop, aboveCount holds the final count of days above the average temperature.
17. printf("Average temperature: %.2f\n", average); prints the calculated average temperature formatted to exactly 2 decimal places using the %.2f format specifier.
18. printf("Days above average: %d\n", aboveCount); prints the count of days that had temperatures above the average using the %d integer format specifier.
19. return 0; signals successful program termination to the operating system.
20. The closing brace } marks the end of the main() function and the program.

Your Output:

Output

```
Enter the 7 daily temperature: 1 2 3 45 6 7 88
Average Temperature: 21.71
Days above average: 2
```

Expected Output:

```
Enter 7 daily temperatures:
28.5 31.2 25.0 30.1 29.8 27.3 32.0
Average temperature: 29.13
Days above average: 3
```

Exercise 2: Grade Distribution Counter

Problem:

Write a C program that reads the marks of N students (maximum 50) into an array and categorizes each mark into one of five grade bands: Distinction (80-100), First Class (60-79), Second Class (45-59), Pass (35-44), and Fail (0-34). The program should count how many students fall into each category and display the distribution.

Code:

```
#include <stdio.h>

int main() {
    int marks[50];
    int n, i;
    int dist = 0, first = 0, second = 0, pass = 0, fail = 0;

    printf("Enter number of students (max 50): ");
    scanf("%d", &n);

    printf("Enter %d marks:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &marks[i]);
    }

    for (i = 0; i < n; i++) {
        if (marks[i] >= 80) {
            dist++;
        } else if (marks[i] >= 60) {
            first++;
        } else if (marks[i] >= 45) {
            second++;
        } else if (marks[i] >= 35) {
            pass++;
        } else {
            fail++;
        }
    }

    printf("\n--- Grade Distribution ---\n");
    printf("Distinction (80-100): %d\n", dist);
    printf("First Class   (60-79): %d\n", first);
    printf("Second Class  (45-59): %d\n", second);
    printf("Pass          (35-44): %d\n", pass);
    printf("Fail          ( 0-34): %d\n", fail);

    return 0;
```

```
}
```

Line-by-Line Explanation:

1. #include <stdio.h> includes the standard I/O library for input and output functions.
2. int main() is the program entry point where execution begins.
3. int marks[50]; declares an integer array named "marks" with a capacity of 50 elements. This array stores the marks of up to 50 students. The array indices range from 0 to 49.
4. int n, i; declares two integer variables: "n" to store the actual number of students (decided at runtime) and "i" as a loop counter variable.
5. int dist = 0, first = 0, second = 0, pass = 0, fail = 0; declares and initializes five counter variables to zero. Each counter tracks the number of students in one grade category: Distinction, First Class, Second Class, Pass, and Fail.
6. printf("Enter number of students (max 50): "); prompts the user to specify how many student marks will be entered. The maximum is limited to 50 by the array size.
7. scanf("%d", &n); reads the integer value entered by the user and stores it in variable n. This determines how many iterations the subsequent loops will perform.
8. printf("Enter %d marks:\n", n); prompts the user to enter exactly n marks, using the value of n in the prompt for clarity.
9. for (i = 0; i < n; i++) { starts a for loop that runs exactly n times. This loop reads each student's mark and stores it sequentially in the marks array.
10. scanf("%d", &marks[i]); reads an integer mark from the keyboard and stores it at index i in the marks array. Each iteration fills the next position in the array.
11. } closes the input loop. Now the marks array contains n valid mark values.
12. for (i = 0; i < n; i++) { starts a second for loop that iterates through all n marks. This loop examines each mark and categorizes it into a grade band using an if-else-if ladder.
13. if (marks[i] >= 80) checks whether the current mark is 80 or above, which qualifies as Distinction. The else-if chain ensures each mark is counted in exactly one category.
14. dist++; increments the Distinction counter when the mark is 80 or above.
15. else if (marks[i] >= 60) is checked only if the previous condition was false (mark < 80). Since we already know marks[i] < 80, this effectively checks the range 60-79 for First Class.
16. first++; increments the First Class counter for marks in the 60-79 range.
17. else if (marks[i] >= 45) checks the range 45-59 for Second Class, only reached if the mark is below 60.
18. second++; increments the Second Class counter for marks in the 45-59 range.
19. else if (marks[i] >= 35) checks the range 35-44 for Pass, only reached if the mark is below 45.
20. pass++; increments the Pass counter for marks in the 35-44 range.
21. else handles the remaining case where marks[i] < 35, which means the student has Failed. No explicit condition is needed because all other possibilities have been exhausted.
22. fail++; increments the Fail counter for marks below 35.
23. } closes the categorization loop. At this point, all five counters hold the correct distribution counts.
24. printf("\n--- Grade Distribution ---\n"); prints a separator header line before the distribution results. The \n at the start creates a blank line for visual separation.
25. printf("Distinction (80-100): %d\n", dist); prints the count of students in the Distinction category.
26-28. The next three printf statements similarly print the counts for First Class, Second Class, and Pass categories, each with its grade range shown in parentheses.

29. printf("Fail (0-34): %d\n", fail); prints the count of students who failed, with the range 0-34 shown. The extra spaces in the range align it visually with the categories above.
30. return 0; signals that the program terminated successfully.
31. The closing brace } ends the main function.

Your Output:

Output

```
Enter numbers of students: 3
Enter 3 Marks:
55 66 77

--- Grade Distribution ---
Distinction (80-100): 0
First Class   (60-79): 2
Second Class  (45-59): 1
Pass          (35-44): 0
Fail          ( 0-34): 0
```

Expected Output:

```
Enter number of students (max 50): 8
Enter 8 marks:
85 72 56 90 33 61 45 28

--- Grade Distribution ---
Distinction (80-100): 2
First Class   (60-79): 2
Second Class  (45-59): 2
Pass          (35-44): 0
Fail          ( 0-34): 2
```

Exercise 3: In-Place Array Reversal Using a Function

Problem:

Write a C program that reads N integers into an array and passes the array to a function called reverseArray(). The function should reverse the elements of the array in-place (without using a second array). Print the array before and after reversal to verify the function works correctly.

Code:

```
#include <stdio.h>

void reverseArray(int arr[], int size) {
    int start = 0, end = size - 1, temp;
    while (start < end) {
        temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
```

```
        start++;
        end--;
    }
}

int main() {
    int arr[100], n, i;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("\nOriginal array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }

    reverseArray(arr, n);

    printf("\nReversed array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Line-by-Line Explanation:

1. #include <stdio.h> includes the standard I/O library for printf() and scanf() functions.
2. void reverseArray(int arr[], int size) defines a function named "reverseArray" that takes two parameters: an integer array "arr" and an integer "size" representing the number of elements. The return type is void because the function modifies the array in-place and does not need to return a value.
3. int start = 0, end = size - 1, temp; declares three local variables: "start" initialized to 0 (pointing to the first element), "end" initialized to size-1 (pointing to the last element), and "temp" used as a temporary holder during swapping.
4. while (start < end) { starts a loop that continues as long as the start index is less than the end index. This ensures the loop stops when the two pointers meet or cross, meaning all necessary swaps are complete.
5. temp = arr[start]; saves the value at the start position in a temporary variable. This is the first step of the classic three-variable swap technique.
6. arr[start] = arr[end]; copies the value from the end position into the start position, overwriting the original value (which was safely saved in temp).
7. arr[end] = temp; places the saved original start value into the end position, completing the swap of the two elements.

8. start++; moves the start pointer one position to the right, advancing toward the center of the array.
9. end--; moves the end pointer one position to the left, also advancing toward the center. Together, start++ and end-- ensure the loop processes pairs from the outside moving inward.
10. } closes the while loop. When the loop finishes, the array has been fully reversed in-place without requiring any additional array storage.
11. } closes the reverseArray function body.
12. int main() is the program entry point where execution begins.
13. int arr[100], n, i; declares an integer array "arr" with capacity for 100 elements, an integer "n" for the actual number of elements, and "i" as a loop counter.
14. printf("Enter number of elements: "); prompts the user to specify how many elements they want to store in the array.
15. scanf("%d", &n); reads the integer value from the user and stores it in n.
16. printf("Enter %d elements:\n", n); prompts the user to enter exactly n integer values.
17. for (i = 0; i < n; i++) { starts a loop to read n elements one by one from the keyboard.
18. scanf("%d", &arr[i]); reads each integer and stores it at position i in the array.
19. printf("\nOriginal array: "); prints a label before displaying the original array contents. The \n creates a blank line for visual separation.
20. for (i = 0; i < n; i++) { starts a loop to print all elements of the original array before reversal.
21. printf("%d ", arr[i]); prints each element followed by a space, displaying the array contents on a single line.
22. reverseArray(arr, n); calls the reverseArray function, passing the array "arr" and its size "n". In C, arrays are passed by reference (actually by pointer), so the function operates on the original array, not a copy. Any changes made inside the function are reflected in main().
23. printf("\nReversed array: "); prints a label before displaying the reversed array contents.
24. for (i = 0; i < n; i++) { starts a loop to print all elements after the reversal has been performed.
25. printf("%d ", arr[i]); prints each element of the now-reversed array, confirming the reversal was successful.
26. printf("\n"); prints a final newline to move the cursor to the next line after the output.
27. return 0; signals successful program termination.
28. The closing brace } ends the main function.

Your Output:

Output

```
Enter number of elements: 5
Enter 5 elements:
1 2 3 4 5

Original array: 1 2 3 4 5
Reversed array: 5 4 3 2 1
```

Expected Output:

```
Enter number of elements: 6
Enter 6 elements:
```

```
10 20 30 40 50 60

Original array: 10 20 30 40 50 60
Reversed array: 60 50 40 30 20 10
```

Exercise 4: Classroom Performance Matrix (2D Array)

Problem:

Write a C program that uses a 2D array (3 students x 4 subjects) to store student marks. The program should calculate and display each student's total marks and average, and also calculate and display the average mark for each subject across all students.

Code:

```
#include <stdio.h>

int main() {
    int marks[3][4];
    int i, j, total;
    float avg;

    printf("Enter marks for 3 students in 4 subjects:\n");
    for (i = 0; i < 3; i++) {
        printf("Student %d: ", i + 1);
        for (j = 0; j < 4; j++) {
            scanf("%d", &marks[i][j]);
        }
    }

    printf("\n--- Student-wise Results ---\n");
    for (i = 0; i < 3; i++) {
        total = 0;
        for (j = 0; j < 4; j++) {
            total += marks[i][j];
        }
        avg = (float)total / 4;
        printf("Student %d - Total: %d, Average: %.2f\n", i + 1, total, avg);
    }

    printf("\n--- Subject-wise Averages ---\n");
    for (j = 0; j < 4; j++) {
        total = 0;
        for (i = 0; i < 3; i++) {
            total += marks[i][j];
        }
        avg = (float)total / 3;
        printf("Subject %d - Average: %.2f\n", j + 1, avg);
    }

    return 0;
}
```

Line-by-Line Explanation:

1. #include <stdio.h> includes the standard I/O library for input/output functions.

2. int main() is the program entry point.
3. int marks[3][4]; declares a 2-dimensional integer array with 3 rows (students) and 4 columns (subjects). The first index (row) represents the student number (0-2), and the second index (column) represents the subject number (0-3). This creates a table-like structure with 12 total elements.
4. int i, j, total; declares three integer variables: "i" for the outer loop (rows/students), "j" for the inner loop (columns/subjects), and "total" to accumulate sums.
5. float avg; declares a floating-point variable to store computed average values. Using float (instead of int) ensures that the average retains decimal precision.
6. printf("Enter marks for 3 students in 4 subjects:\n"); prints an instruction line telling the user to enter the mark data.
7. for (i = 0; i < 3; i++) { starts the outer for loop that iterates through each student (rows 0, 1, and 2). Each iteration handles one student's data entry.
8. printf("Student %d: ", i + 1); prints a prompt for the current student. i + 1 is used because arrays are zero-indexed but students are naturally numbered starting from 1.
9. for (j = 0; j < 4; j++) { starts the inner for loop that iterates through each subject (columns 0, 1, 2, and 3) for the current student.
10. scanf("%d", &marks[i][j]); reads an integer from the keyboard and stores it in the 2D array at row i, column j. The two indices together pinpoint the exact memory location within the 2D array.
11. } closes the inner loop (subjects). After this loop, all 4 marks for student i have been entered.
12. } closes the outer loop (students). At this point, the entire 3x4 matrix is filled with data.
13. printf("\n--- Student-wise Results ---\n"); prints a section header. The \n creates a blank line before the header for visual separation from the input section.
14. for (i = 0; i < 3; i++) { starts a loop to process each student's results. For each student, it will calculate the total and average of their 4 subject marks.
15. total = 0; resets the accumulator to zero before processing each student. This is critical; without it, total would carry over values from the previous student.
16. for (j = 0; j < 4; j++) { starts the inner loop to sum all 4 subject marks for the current student.
17. total += marks[i][j]; adds the mark at position [i][j] to the running total. After this inner loop completes, total holds the sum of all 4 subjects for student i.
18. } closes the inner summation loop.
19. avg = (float)total / 4; divides the total by 4 to compute the average. The (float) type cast is essential here; without it, integer division would truncate the result (e.g., 285/4 = 71 instead of 71.25).
20. printf("Student %d - Total: %d, Average: %.2f\n", i + 1, total, avg); prints the student number, total marks, and average for the current student. %.2f formats the average to 2 decimal places.
21. } closes the student-wise results loop.
22. printf("\n--- Subject-wise Averages ---\n"); prints a header for the subject analysis section. This time, the program will calculate the average of each subject across all students.
23. for (j = 0; j < 4; j++) { starts a loop that iterates through each subject (columns). Notice the loop order is reversed here: the outer loop is over columns (subjects) and the inner loop will be over rows (students).
24. total = 0; resets the accumulator to zero before processing each subject.
25. for (i = 0; i < 3; i++) { starts the inner loop that iterates through each student (rows) for the current subject. This accesses the same column across all rows.

26. total += marks[i][j]; adds the mark of student i in subject j to the running total. After this loop, total holds the sum of all 3 students' marks in subject j.
27. } closes the inner student loop.
28. avg = (float)total / 3; divides the column sum by 3 (number of students) to compute the subject average. The (float) cast ensures decimal precision.
29. printf("Subject %d - Average: %.2f\n", j + 1, avg); prints the subject number and its average mark across all students, formatted to 2 decimal places.
30. } closes the subject-wise averages loop.
31. return 0; signals successful program termination.
32. The closing brace } ends the main function.

Your Output:

Output

```
Enter marks for 3 students in 4 subjects:
Student 1: 56 58 75 99
Student 2: 256 50 75 100
Student 3: 1 23  4 5

--- Student-wise Results ---
Student 1 - Total: 288, Average: 72.00
Student 2 - Total: 481, Average: 120.25
Student 3 - Total: 33, Average: 8.25

--- Subject-wise Averages ---
Subject 1 - Average: 104.33
Subject 2 - Average: 43.67
Subject 3 - Average: 51.33
Subject 4 - Average: 68.00
```

Expected Output:

```
Enter marks for 3 students in 4 subjects:
Student 1: 78 85 62 90
Student 2: 55 70 88 76
Student 3: 92 81 74 68

--- Student-wise Results ---
Student 1 - Total: 315, Average: 78.75
Student 2 - Total: 289, Average: 72.25
Student 3 - Total: 315, Average: 78.75

--- Subject-wise Averages ---
Subject 1 - Average: 75.00
Subject 2 - Average: 78.67
Subject 3 - Average: 74.67
Subject 4 - Average: 78.00
```

Exercise 5: Voter ID Verification Using Sequential Search

Problem:

Write a C program that stores a list of 10 registered voter IDs in an array. The program should then ask the user to enter a voter ID to search for. Implement sequential (linear) search to determine whether the ID exists in the registered list. If found, display its position (index + 1); otherwise, display a not found message.

Code:

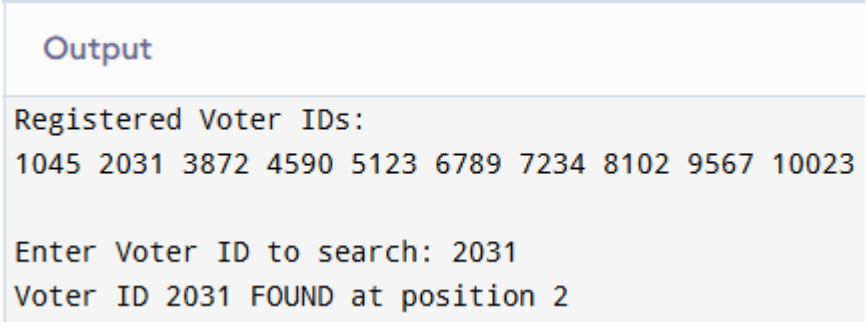
```
#include <stdio.h>

int main() {
    int voterIDs[10] = {1045, 2031, 3872, 4590, 5123,
                        6789, 7234, 8102, 9567, 10023};
    int searchID, i, found = 0, position = -1;

    printf("Registered Voter IDs:\n");
    for (i = 0; i < 10; i++) {
        printf("%d ", voterIDs[i]);
    }

    printf("\n\nEnter Voter ID to search: ");
    scanf("%d", &searchID);

    for (i = 0; i < 10; i++) {
        if (voterIDs[i] == searchID) {
            found = 1;
            position = i;
            break;
        }
    }

    if (found) {
        printf("Voter ID %d FOUND at position %d\n", searchID, position + 1);
    } else {
        printf("Voter ID %d NOT FOUND in the list.\n", searchID);
    }

    return 0;
}
```

Line-by-Line Explanation:

1. #include <stdio.h> includes the standard I/O library for printf() and scanf() functions.
2. int main() is the program entry point where execution begins.
3. int voterIDs[10] = {1045, 2031, 3872, 4590, 5123, 6789, 7234, 8102, 9567, 10023}; declares and initializes an integer array with 10 pre-registered voter IDs. The values are provided at declaration time using an initializer list, so each element gets its value when the array is created in memory.
4. int searchID, i, found = 0, position = -1; declares four integer variables: "searchID" to store the ID the user wants to find, "i" as a loop counter, "found" initialized to 0 (acting as a boolean flag: 0 = not found, 1 = found), and "position" initialized to -1 (an invalid index that will be overwritten if the ID is found).

5. printf("Registered Voter IDs:\n"); prints a header label before displaying the list of all registered IDs so the user can see what IDs are available to search for.
6. for (i = 0; i < 10; i++) { starts a for loop to iterate through all 10 positions in the voterIDs array, printing each one.
7. printf("%d ", voterIDs[i]); prints the voter ID at index i followed by a space, displaying all registered IDs on a single line.
8. } closes the display loop.
9. printf("\n\nEnter Voter ID to search: "); prints two newlines (one blank line for separation and one for the prompt line) followed by the search prompt.
10. scanf("%d", &searchID); reads the integer entered by the user and stores it in the searchID variable. This is the value the sequential search will look for in the array.
11. for (i = 0; i < 10; i++) { starts the sequential search loop. Sequential search examines each element from the first index (0) to the last (9), one at a time, comparing each with the target value.
12. if (voterIDs[i] == searchID) compares the element at the current index i with the searchID. If they match, it means the voter ID has been found in the list.
13. found = 1; sets the found flag to 1 (true), indicating a successful search. This flag will be checked after the loop to determine what message to display.
14. position = i; records the current index where the match was found. This allows the program to report exactly where in the list the ID was located.
15. break; immediately exits the for loop. Since sequential search only needs to find one match, there is no need to continue checking remaining elements once the target is found. This also improves efficiency.
16. } closes the if block.
17. } closes the search loop. If the loop completes without hitting the break statement, it means the voter ID was not found in the array, and found remains 0.
18. if (found) { checks the boolean flag. In C, any non-zero value is considered true, so if found == 1, the ID was located in the array.
19. printf("Voter ID %d FOUND at position %d\n", searchID, position + 1); prints a success message showing the searched ID and its 1-based position. position + 1 converts from the zero-based array index to a human-readable position number.
20. } closes the if block.
21. else handles the case where found is still 0, meaning the entire array was searched without finding a match.
22. printf("Voter ID %d NOT FOUND in the list.\n", searchID); prints a failure message informing the user that the entered voter ID does not exist in the registered list.
23. } closes the else block.
24. return 0; signals successful program termination.
25. The closing brace } ends the main function.

Your Output:



Expected Output:

```
Registered Voter IDs:
1045 2031 3872 4590 5123 6789 7234 8102 9567 10023

Enter Voter ID to search: 7234
Voter ID 7234 FOUND at position 7
```

Exercise 6: Bubble Sort Scoreboard

Problem:

Write a C program that reads N cricket player scores into an array and sorts them in descending order using the Bubble Sort algorithm. The program should display the scores before and after sorting. Also display the highest and lowest scores.

Code:

```
#include <stdio.h>

int main() {
    int scores[50];
    int n, i, j, temp;

    printf("Enter number of players: ");
    scanf("%d", &n);

    printf("Enter %d scores:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &scores[i]);
    }

    printf("\nBefore sorting: ");
    for (i = 0; i < n; i++) {
        printf("%d ", scores[i]);
    }

    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - 1 - i; j++) {
            if (scores[j] < scores[j + 1]) {
                temp = scores[j];
                scores[j] = scores[j + 1];
                scores[j + 1] = temp;
            }
        }
    }

    printf("\nAfter sorting (descending): ");
    for (i = 0; i < n; i++) {
        printf("%d ", scores[i]);
    }

    printf("\nHighest score: %d", scores[0]);
    printf("\nLowest score: %d\n", scores[n - 1]);
```

```
    return 0;
}
```

Line-by-Line Explanation:

1. #include <stdio.h> includes the standard I/O library for printf() and scanf() functions.
2. int main() is the program entry point where execution begins.
3. int scores[50]; declares an integer array named "scores" with a capacity of 50 elements. This array will store the cricket scores of up to 50 players.
4. int n, i, j, temp; declares four integer variables: "n" for the actual number of players, "i" and "j" as loop counters for the nested sorting loops, and "temp" as a temporary variable used during the swap operation in Bubble Sort.
5. printf("Enter number of players: "); prompts the user to enter the number of players whose scores will be recorded.
6. scanf("%d", &n); reads the integer from the keyboard and stores it in n.
7. printf("Enter %d scores:\n", n); prompts the user to enter exactly n score values.
8. for (i = 0; i < n; i++) { starts a for loop that runs n times to read each player's score from the keyboard.
9. scanf("%d", &scores[i]); reads an integer score and stores it at position i in the scores array.
10. } closes the input loop. The array now contains n unsorted score values.
11. printf("\nBefore sorting: "); prints a label before displaying the original unsorted array.
12. for (i = 0; i < n; i++) { starts a loop to print all elements of the unsorted array.
13. printf("%d ", scores[i]); prints each score followed by a space on the same line.
14. } closes the display loop.
15. for (i = 0; i < n - 1; i++) { starts the outer loop of the Bubble Sort algorithm. This loop controls the number of passes. After each pass, the next largest element bubbles up to its correct position at the top. n-1 passes are sufficient because after n-1 passes, all n elements are guaranteed to be in order.
16. for (j = 0; j < n - 1 - i; j++) { starts the inner loop that performs pairwise comparisons and swaps within each pass. The upper limit is n-1-i because after i passes, the last i elements are already sorted and do not need to be checked again. This optimization reduces unnecessary comparisons.
17. if (scores[j] < scores[j + 1]) compares adjacent elements. For descending order, if the current element is less than the next element, they need to be swapped so the larger value moves toward the beginning of the array.
18. temp = scores[j]; saves the value at position j in a temporary variable. This is the first step of the three-variable swap.
19. scores[j] = scores[j + 1]; copies the next element's value into the current position, overwriting the saved value.
20. scores[j + 1] = temp; places the saved value into the next position, completing the swap. The larger value has now moved one position closer to the front of the array.
21. } closes the inner if block.
22. } closes the inner loop (comparisons within a single pass).
23. } closes the outer loop (all passes). After all passes complete, the array is sorted in descending order.
24. printf("\nAfter sorting (descending): "); prints a label before displaying the sorted array.
25. for (i = 0; i < n; i++) { starts a loop to print all elements of the sorted array.
26. printf("%d ", scores[i]); prints each sorted score followed by a space.

27. } closes the display loop.
28. printf("\nHighest score: %d", scores[0]); prints the highest score. Since the array is sorted in descending order, the first element (index 0) is the largest value.
29. printf("\nLowest score: %d\n", scores[n - 1]); prints the lowest score. In a descending-order sorted array, the last element (index n-1) is the smallest value.
30. return 0; signals successful program termination.
31. The closing brace } ends the main function.

Your Output:

Output

```
Enter number of players: 11
Enter 11 scores:
22 33 44 55 66 77 88 99 11 10 26

Before sorting: 22 33 44 55 66 77 88 99 11 10 26
After sorting (descending): 99 88 77 66 55 44 33 26 22 11 10
Highest score: 99
Lowest score: 10
```

Expected Output:

```
Enter number of players: 6
Enter 6 scores:
45 89 23 76 56 91

Before sorting: 45 89 23 76 56 91
After sorting (descending): 91 89 76 56 45 23
Highest score: 91
Lowest score: 23
```